

FUTURE SCALE PREDICTION USING XG BOOST

Future Sales Prediction using XGBoost

This section implements a regression model to predict future sales based on historical trends.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error

# 1. Generate Synthetic Sales Data
np.random.seed(42)
dates = pd.date_range(start='2022-01-01', end='2023-12-31', freq='D')
sales = 100 + 2 * np.arange(len(dates)) + 50 * np.sin(np.arange(len(dates)) * (2 * np.pi / 30)) + np.random.randn(len(dates))

df_sales = pd.DataFrame({'Date': dates, 'Sales': sales})
df_sales.set_index('Date', inplace=True)

display(df_sales.head())
```

	Sales
Date	
2022-01-01	104.967142
2022-01-02	111.012942
2022-01-03	130.813718
2022-01-04	150.619561
2022-01-05	142.815708

```
[16] ✓ Os
# 2. Feature Engineering (Lags & Rolling Windows)
def create_features(df):
    df = df.copy()
    df['dayofweek'] = df.index.dayofweek
    df['quarter'] = df.index.quarter
    df['month'] = df.index.month
    df['year'] = df.index.year
    # Lag features
    for i in [1, 7, 30]:
        df[f'lag_{i}'] = df['Sales'].shift(i)
    # Rolling mean
    df['rolling_mean_7'] = df['Sales'].shift(1).rolling(window=7).mean()
    return df

df_featured = create_features(df_sales).dropna()

# Split into Train and Test (last 60 days for testing)
train = df_featured.iloc[:-60]
test = df_featured.iloc[-60:]

X_train = train.drop(columns=['Sales'])
y_train = train['Sales']
X_test = test.drop(columns=['Sales'])
y_test = test['Sales']

# Display the engineered features
print("Engineered Features (First 5 rows):")
display(df_featured.head())
```

.. Engineered Features (First 5 rows):

	Sales	dayofweek	quarter	month	year	lag_1	lag_7	lag_30	rolling_mean_7
Date									
2022-01-31	153.982934	0	1	1	2022	144.687478	82.026423	104.967142	112.983218
2022-02-01	190.918366	1	1	2	2022	153.982934	95.003347	111.012942	123.262719
2022-02-02	184.201860	2	1	2	2022	190.918366	107.807956	130.813718	136.964865
2022-02-03	184.812153	3	1	2	2022	184.201860	103.332823	150.619561	147.878280
2022-02-04	213.382690	4	1	2	2022	184.812153	128.367718	142.815708	159.518184

```
# 3. Model Training
model = XGBRegressor(n_estimators=1000, learning_rate=0.05, max_depth=5)
model.fit(X_train, y_train, eval_set=[(X_train, y_train), (X_test, y_test)], verbose=False)

# 4. Prediction and Visualization
test['Predictions'] = model.predict(X_test)

plt.figure(figsize=(12, 6))
plt.plot(train.index[-100:], train['Sales'][-100:], label='Historical Sales')
plt.plot(test.index, test['Sales'], label='Actual Sales (Test)')
plt.plot(test.index, test['Predictions'], label='Predicted Sales', linestyle='--')
plt.title('Future Sales Prediction with XGBoost')
plt.legend()
plt.show()

rmse = np.sqrt(mean_squared_error(test['Sales'], test['Predictions']))
print(f'Test RMSE: {rmse:.2f}')
```

... /tmp/ipykernel_3610/2977780102.py:6: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html

```
test['Predictions'] = model.predict(X_test)
```

Future Sales Prediction with XGBoost

